CODEALPHA · TASK 3

Handwritten Character Recognition

Chronological model development, error analysis and evaluation

This report documents the development of a single-character classifier using EMNIST Letters. It follows the experiments in the conversation record and Google Colab export, including the decisions that followed each result, implementation problems, diagnostic work, and the final benchmark evaluation.

Scope. Dataset preparation, CNN architecture, training and regularisation experiments, checkpoint management, error analysis, image preprocessing, and evaluation. Unrelated deployment and interface work is excluded.

BEST OFFICIAL TEST ACCURACY	MACRO F1	BEST VALIDATION ACCURACY
95.64%	**95.65%**	**95.74%**
VGG-style CNN; 19,893 / 20,800	26 alphabet classes	VGG-style checkpoint; epoch 30

Prepared from the supplied project-history PDF, Google Colab notebook export, and the follow-up research notes. Numerical results are reported as recorded in those sources; validation and test metrics are kept distinct.

Abstract

A convolutional neural network (CNN) was developed for 26-way lowercase handwritten-character classification using EMNIST Letters. The initial custom CNN achieved 93.14% test accuracy. Subsequent experiments investigated training curves, confusion patterns, batch normalisation, spatial augmentation, dropout, image orientation, checkpoint handling, and a deeper VGG-style CNN. Error analysis showed that mistakes were concentrated in visually similar characters, particularly i/l and g/q. The selected VGG-style checkpoint achieved 95.64% accuracy and 95.65% macro F1 on the standard 20,800-image test split. A separate photographed-handwriting trial demonstrated that preprocessing mismatch can sharply reduce performance even when benchmark accuracy is high. The report therefore treats benchmark classification and real-image inference as related but distinct evaluation problems.

1. Dataset and evaluation protocol

The project uses **EMNIST Letters**, a 28 × 28 grayscale character dataset derived from NIST Special Database 19. The torchvision split contains 124,800 training images and 20,800 test images. Each of the 26 letters contributes 800 test examples. This makes the benchmark appropriate for controlled single-character classification, but not a direct proxy for photographs of handwriting on paper [1].

Property	Project configuration
Dataset API	torchvision.datasets.EMNIST(split="letters")
Input tensor	1 × 28 × 28 grayscale image
Official training split	124,800 images
Official test split	20,800 images; 800 per letter
Classes	a–z; the original pipeline used 27 outputs with class 0 unused, later VGG model used 26 outputs

The first implementation retained the dataset's label convention (labels 1–26 and an unused N/A class at index 0). This was not responsible for all observed errors, but it complicated metric interpretation: a classification report that includes an unsupported class can depress macro averages. Later evaluation explicitly reported the 26 actual letters.

The original EMNIST paper describes the dataset construction and benchmark format; it was used here to justify dataset choice and to interpret this as a standardised benchmark rather than unrestricted handwriting recognition [1].

2. Initial CNN and baseline establishment

The first model was a custom CNN implemented in PyTorch. It used four convolutional stages with 8, 16, 32 and 64 output channels, batch normalisation, ReLU activations and max pooling, followed by a linear classification layer. The training loop used Adam with a learning rate of 0.001 and cross-entropy loss. The original experiment used batch size 32 and 32 epochs.

Run / observation	Recorded result	Interpretation and decision
Initial baseline	95.45% train; 93.14% test	A useful first benchmark, but a visible train–test gap suggested limited generalisation.
Later baseline runs	Approximately 93.7–94.0% test	Accuracy plateaued; more epochs alone did not reliably solve the remaining errors.
Learning curves	Train accuracy continued rising while test accuracy flattened near 94%	Consistent with a moderate generalisation gap, not evidence of severe overfitting by itself.

At this stage, the main issue was not whether the model could learn the dataset—it could—but why errors persisted. The next work shifted from architecture changes to collecting diagnostic evidence: loss/accuracy curves, a confusion matrix, per-class precision/recall/F1, and examples of incorrect predictions.

A code-structure issue was also corrected during metric plotting: evaluation had to run once after each epoch, not inside the mini-batch loop. Otherwise the stored curves would represent repeated partial-epoch measurements and would not be comparable across epochs.

3. First error analysis: confusion matrix and per-class metrics

The confusion matrix and classification report showed that aggregate accuracy concealed uneven class performance. The earliest report showed relatively weak F1 for i, l, g and q; the strongest recurring pattern was confusion between i and l, followed by g and q. These were not isolated visual mistakes: they recurred across later experiments.

The confusion matrix was used as a diagnostic tool rather than a decorative result. Its diagonal represents correct predictions; off-diagonal cells identify which actual characters the model systematically maps to another class. Precision and recall helped distinguish false-positive predictions from missed examples, while F1 summarised the balance between the two.

The EMNIST paper and later independent evaluation both discuss the difficulty of character-level recognition and benchmark performance; in this project, these sources provided context for interpreting similar-letter errors rather than assuming the entire pipeline was defective [1, 2].

4. Controlled experiments on the original CNN

Changes were introduced as separate experiments so their effect could be compared with the baseline. The project history records the following sequence; not every change produced a reliable gain.

Experiment	What changed	Observed outcome / decision
Batch normalisation	Added normalisation after convolutional layers	A small accuracy increase (about one percentage point was observed in an early run); retained in later models.
Training augmentation	RandomAffine with rotation up to 10° and translation up to 10%	Intended to reduce sensitivity to small shifts and tilt. Results varied and remained around the mid-94% range.
Milder translation	Reduced translation to 5% while retaining small rotations	Changed the i/l error balance but did not eliminate the dominant confusion.
Dropout experiment	Trained a separate model with dropout	Essentially tied with the best mild-augmentation run; not a decisive improvement.
Checkpoint saving	Saved state dictionaries separately for baseline and experimental models	Avoided overwriting the original model and made comparisons reproducible. A variable-name mismatch (`model` vs `model_aug`) initially caused a save/load confusion.
Horizontal flipping	Considered as an augmentation	Not adopted: flipping letters can change their identity or produce unrealistic samples.

The training augmentation was applied only to training images; evaluation images were not augmented. This distinction matters because the test distribution should remain stable. The experiment did not support a claim that augmentation alone solved generalisation. Instead, it showed that small spatial perturbations affected the error distribution but left the hardest letter pairs largely unresolved.

Checkpoint and notebook-state issues were practical but important. In Colab, runtime variables do not persist reliably across disconnected sessions, and a missing variable is not evidence that a saved checkpoint is lost. The fix was to load a named checkpoint into the matching architecture and keep baseline and experimental files separate.

5. Orientation mismatch: a consequential preprocessing finding

A later diagnostic review found that EMNIST images needed an orientation correction. A transpose operation was added to the preprocessing pipeline, and training and validation transforms were checked for consistency. This was a substantive correction: when the model is trained on one orientation and evaluated on another, it sees a shifted input distribution and can confuse characters whose shapes are orientation-sensitive.

The implementation was inspected rather than guessed: the test transform's `Lambda` was traced to `Image.Transpose.TRANSPOSE`, confirming that rows and columns were swapped. Both training and evaluation needed the same operation. The correction improved consistency, but the remaining errors still clustered around i/l and g/q, so orientation was not treated as the sole explanation.

6. Validation diagnostics and training-schedule review

A later CNN checkpoint was retrained from scratch using a seeded split of the original training set: 112,320 training images and 12,480 validation images. The saved checkpoint was `emnist_cnn_improved_val_best.pth`. The output layer had 27 logits because the original label mapping used indices 1–26, leaving class 0 unused.

Epoch	Train accuracy	Validation accuracy	Validation loss
1	83.05%	93.19%	0.1967
2	90.94%	94.03%	0.1734
3	92.06%	94.48%	0.1603
4	92.70%	94.25%	0.1633
5	93.20%	94.94%	0.1505
6	93.49%	94.89%	0.1456
7	93.67%	94.86%	0.1515

Validation accuracy peaked at 94.94% at epoch 5, while validation loss reached its minimum at epoch 6 (0.1456). Early stopping ended training after epoch 7. Because the scheduler reduced the learning rate near the end of this run, the review noted that the model had little opportunity to benefit from the reduced rate. This was recorded as a hypothesis to test—not as a proven cause of the remaining errors.

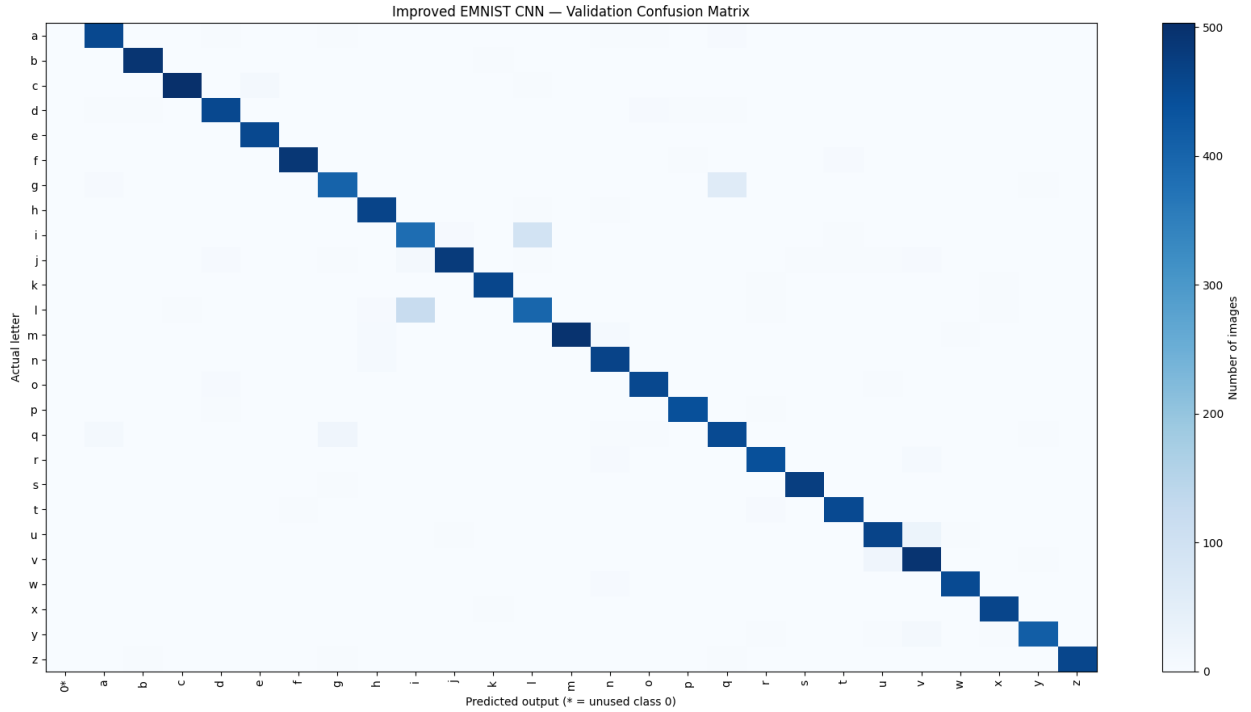


Figure 1. Validation confusion matrix for the 94.94% CNN checkpoint. The diagonal dominates, but systematic off-diagonal errors remain.

The validation run made the error pattern quantifiable: 11,848/12,480 correct and 632 incorrect. Per-letter accuracy was lowest for l (74.53%), i (78.85%) and g (85.59%). The combined i↔l, g↔q and u↔v pairs accounted for 340/632 errors (53.8%). This evidence motivated benchmarking a stronger CNN architecture rather than adding further small regularisation changes without a clear hypothesis.

7. Research-informed architecture change

A literature check was used to choose a targeted next experiment. The 2026 Scientific Data study by Szyma■ski and colleagues independently evaluated VGG-5, VGG-5 with SpinalFC, TextCaps and WaveMixLite on EMNIST Letters, reproducing VGG-5 at 95.87% and WaveMixLite at 95.69%. The same study found substantially lower scores on a separately collected handwriting dataset, demonstrating that benchmark performance does not guarantee generalisation to new handwriting styles [2].

The practical decision was to benchmark a VGG-style CNN because it remained within the CNN scope of the assignment and offered a concrete, published architecture family to compare against. The source was used as an architecture-selection reference and as a warning to evaluate real handwriting separately; published scores were not treated as directly comparable unless the evaluation split matched.

The VGG-style experiment used repeated 3 × 3 convolution blocks with batch normalisation, ReLU and max pooling, followed by a dropout-based classifier with 26 outputs. Training used AdamW (learning rate 0.001, weight decay 0.0001), a learning-rate scheduler and early stopping. The best checkpoint was saved at epoch 30.

Checkpoint metric	Original CNN	VGG-style CNN
Best validation accuracy	94.94%	95.74%
Training accuracy at best checkpoint	93.20% (epoch 5 for prior run)	97.21%
Validation loss at best checkpoint	0.1505	0.1328
Decision	Retain as baseline	Promote to selected checkpoint

The VGG-style model improved validation accuracy by 0.80 percentage points, roughly 100 additional correct classifications on a 12,480-image validation set. Training accuracy was higher than validation accuracy, so a generalisation gap remained, but the validation score and loss both improved over the previous CNN checkpoint.

8. Official test evaluation of the selected checkpoint

The selected checkpoint, `emnist_vgg_benchmark_val_best.pth`, was loaded without retraining and evaluated on the 20,800-image official EMNIST test split. The run used CUDA in Colab. This separated checkpoint selection (validation set) from the reported benchmark measurement (test set), although the project history shows the official test split had been inspected during earlier development; the final score should therefore be interpreted as a strong estimate, not a completely untouched one.

TEST ACCURACY	MACRO F1	WEIGHTED F1	TEST LOSS
95.64%	**95.65%**	**95.65%**	**0.1403**
19,893 / 20,800 correct	Across 26 letters	Support-weighted	Cross-entropy loss

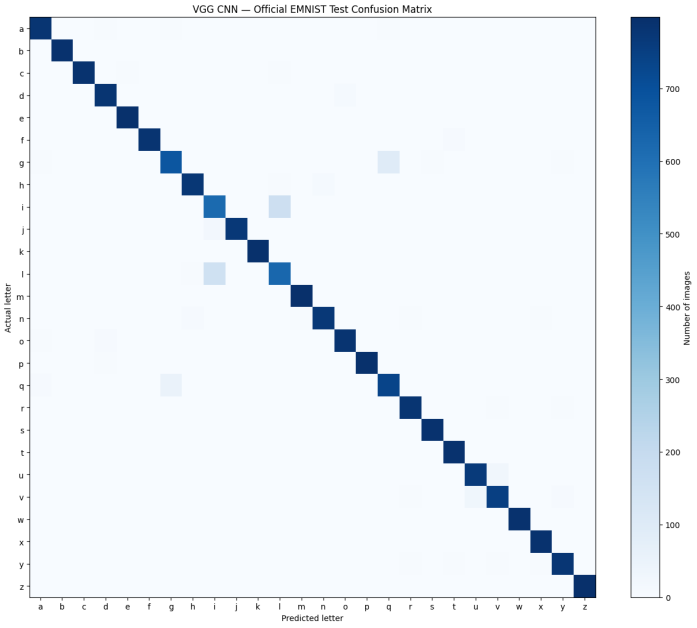


Figure 2. Confusion matrix from the VGG-style CNN on the official EMNIST test split.

The test report recorded 907 incorrect predictions. Most classes achieved high precision, recall and F1; however, class-level results showed that the overall score was not uniform. Test accuracy was 77.25% for i, 78.38% for l, 85.25% for g and 91.88% for q. The i/l pair alone accounted for a substantial share of the errors.

9. Residual error analysis

The most frequent directional errors recorded for the selected checkpoint were i→l (171), l→i (161), g→q (96), q→g (50), v→u (31) and u→v (26). The two main bidirectional pairs, i↔l and g↔q, accounted for 478 of 907 mistakes. The confusion matrix therefore identified a concentrated error pattern rather than broad failure across all classes.

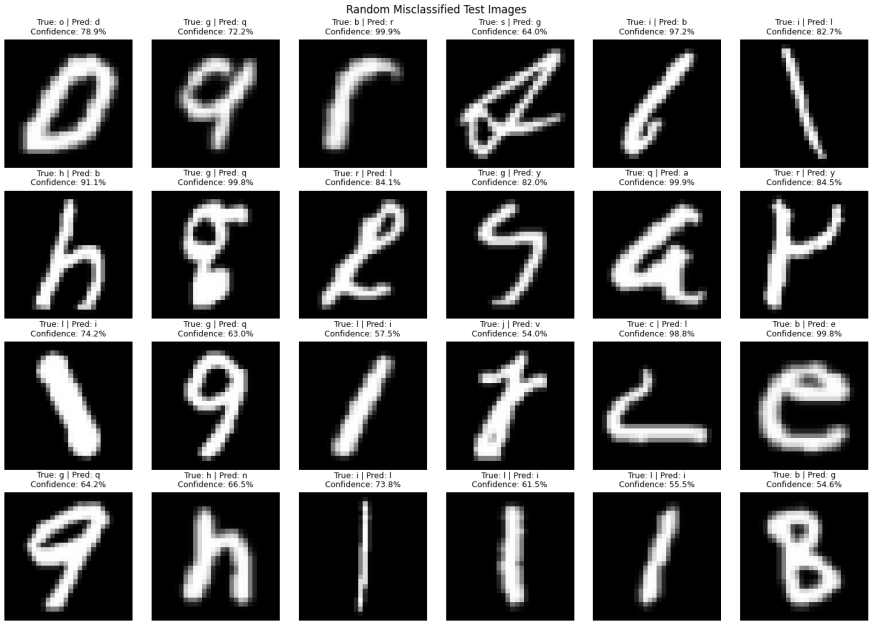


Figure 3. Examples of incorrect test predictions. The examples illustrate the visual similarity that underlies some of the dominant confusions.

This distinction shaped the next decision: the model was not retrained indefinitely in pursuit of a marginal increase. The measured result was already within the 95–96% target range identified from the literature review. The remaining errors were documented as limitations. The 2026 independent study's EMNIST-versus-independent-handwriting gap reinforced the need to separate standardised benchmark accuracy from performance on photographed inputs [2].

10. Real handwriting: inference and preprocessing failures

The saved model was then tested on photographs of single handwritten characters. The first upload attempt selected an EMNIST binary label file (`.idx1-ubyte`) instead of an image; this was an input-format error, not a model failure. The inference path was corrected to accept image formats such as PNG/JPG. Early photo predictions were inconsistent because the photos contained paper texture, ruled lines, uneven illumination, variable margins and stroke thickness that were absent or differently represented in the 28 × 28 benchmark images.

Several debugging steps were required. First, the exact tensor shown to the model was visualised, exposing loss of fine strokes and mismatch in character placement. Second, the preprocessing was traced back to the training transform to preserve orientation. Third, crop, contrast and gamma adjustments were compared. Gamma enhancement improved one handwritten g example from 39.09% to 88.39% confidence, but the effect was not reliable across all letters; it was therefore not adopted as a universal fix. Aggressive thresholding also fragmented strokes and was rejected.

11. Iterative preprocessing and what the results mean

Inference variant	Recorded result	Interpretation
Initial real-image path	4 / 26 correct (15.38%)	Large domain/preprocessing mismatch; not a fair measure of EMNIST benchmark accuracy.
Improved paper/ink handling	22 / 26 (84.62%)	Cropping and contrast handling materially improved the input.
Method C: paper detection, ink extraction, centring and resizing	25 / 26 (96.15%)	A more consistent character crop made the input resemble the training format.
Method C + conservative i/l rule	26 / 26 (100%)	Perfect on this small, curated sample only; rule-based correction means it is not a pure CNN result.

The 26/26 result is not a generalisation estimate: the sample is small, was iteratively inspected, and includes a rule-based correction for the visually ambiguous i/l pair. It should be reported as a successful demonstration on the selected examples, not as evidence of 100% real-world accuracy. The 95.64% EMNIST test score remains the principal benchmark result.

The real-image work also revealed practical implementation errors: one cell assumed a model class name that was not defined (`EMNISTCNN`), another attempted to call `.convert()` on raw uploaded bytes rather than a PIL image, and earlier preprocessing nearly erased a handwritten stroke. These were fixed by using the actual model class/checkpoint, opening uploaded bytes as image objects, and inspecting intermediate crops before prediction.

12. Summary of decisions and current status

Stage	Evidence-led conclusion
Baseline CNN	Functional first model; 93.14% initial test accuracy.
Batch normalisation / augmentation / dropout	Useful controlled experiments, but none alone resolved the main confusion pairs.
Orientation and validation diagnostics	Found and corrected preprocessing inconsistency; isolated i/l, g/q and u/v as dominant errors.
VGG-style CNN	Best checkpoint; 95.74% validation and 95.64% official test accuracy.
Real photographed handwriting	Performance depended strongly on cropping and input normalisation; benchmark accuracy did not transfer automatically.
Current conclusion	Retain the VGG-style checkpoint and document residual errors. Do not claim perfect general-purpose handwriting recognition.

A limitation of the current record is that the official test set was queried during multiple model iterations. For strict research evaluation, the final model should ideally be selected using validation data and tested once on an untouched test set. Also, the custom handwriting set was small and not independently collected at scale.

References

[1] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, “EMNIST: an extension of MNIST to handwritten letters,” arXiv:1702.05373, 2017. Available: https://arxiv.org/abs/1702.05373.

How it was used: cited as the primary source for the EMNIST dataset, its derivation from NIST Special Database 19, and its role as a standardised handwritten-character benchmark.

[2] J. Szyma■ski, K. Skar■y■ski, B. Szutenberg, K. Ratkowska, and colleagues, “The dataset for extending EMNIST evaluation,” *Scientific Data*, vol. 13, article 73, published 22 January 2026. DOI: 10.1038/s41597-025-06291-z.

How it was used: the reported VGG-5 and WaveMixLite EMNIST results informed the decision to benchmark a VGG-style CNN. Its independent handwriting evaluation (VGG-5 84.27% versus 95.87% on EMNIST) was used to contextualise the observed gap between standard test images and photographs of user handwriting. Those scores are contextual benchmarks, not direct guarantees for this implementation.

[3] NIST, “The EMNIST Dataset,” dataset documentation. Available: https://www.nist.gov/itl/products-and-services/emnist-dataset.

How it was used: verified the EMNIST Letters split description and the recommended citation for use of the dataset.

Source and reporting note

The chronological account and experimental metrics were reconstructed from the supplied project-history PDF, the Google Colab export, and the research-summary PDF. Where outputs gave different numbers for different checkpoints, the report preserves the checkpoint and split labels rather than merging them into a single score. The VGG result is reported as 95.64% official test accuracy; the earlier 94.94% result is validation accuracy; and the 26/26 photographed-handwriting result is kept separate from both.

Reproducibility note

Primary checkpoint names in the supplied records: `emnist_cnn_improved_val_best.pth` and `emnist_vgg_benchmark_val_best.pth`. Experiments were conducted in Google Colab with PyTorch/torchvision; the final VGG evaluation reported CUDA as the device. This report does not claim independent reproduction beyond the outputs present in the supplied files.